

Import Libraries

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
import random
import json

import warnings
warnings.filterwarnings('ignore')
```

Data Collection:

```
with open('Intent.json', 'r') as f:
    data = json.load(f)
```

```
print(data.keys())
print(type(data['intents']))
print(len(data['intents']))
print(data['intents'][0].keys())
data['intents'][-1]
```

```
dict_keys(['intents'])
<class 'list'>
22
dict_keys(['intent', 'text', 'responses', 'extension', 'context', 'entityType', 'entities'])
{'intent': 'SelfAware',
 'text': ['Can you prove you are self-aware',
 'Can you prove you are self aware',
 'Can you prove you have a conscious',
 'Can you prove you are self-aware please',
 'Can you prove you are self aware please',
 'Can you prove you have a conscious please',
 'prove you have a conscious'],
 'responses': ['That is an interesting question, can you prove that you are?',
 'That is an difficult question, can you prove that you are?',
 'That depends, can you prove that you are?'],
 'extension': {'function': '', 'entities': False, 'responses': []},
 'context': {'in': '', 'out': '', 'clear': False},
 'entityType': 'NA',
 'entities': []}
```

Data Cleaning:

```
def clean(line):
    cleaned_line = ''
    for char in line:
        if char.isalpha():
            cleaned_line += char
        else:
            cleaned_line += ' '
    cleaned_line = ' '.join(cleaned_line.split())
    return cleaned_line
```

Data Preprocessing:

```

#list of intents
intents = []
unique_intents = []
#all text data to create a corpus
text_input= []
#dictionary mapping intent with appropriate response
response_for_intent = {}
for intent in data['intents']:
    #list of unique intents
    if intent['intent'] not in unique_intents:
        unique_intents.append(intent['intent'])
    for text in intent['text']:
        #cleaning is done before adding text to corpus
        text_input.append(clean(text))
        intents.append(intent['intent'])
    if intent['intent'] not in response_for_intent:
        response_for_intent[intent['intent']] = []
    for response in intent['responses']:
        response_for_intent[intent['intent']].append(response)

```

```

print("Intent :",intents[0])
print("Number of Intent:",len(intents))
print("Sample Input:", text_input[0])
print('Length of text_input:',len(text_input))
print("Sample Response: ", response_for_intent[intents[0]])

```

```

Intent : Greeting
Number of Intent: 143
Sample Input: Hi
Length of text_input: 143
Sample Response: ['Hi human, please tell me your GeniSys user', 'Hello human, please tell me your Gen

```

Tokenization and Embedding

```

tokenizer = Tokenizer(filters='',oov_token='<unk>')
tokenizer.fit_on_texts(text_input)
sequences = tokenizer.texts_to_sequences(text_input)
padded_sequences = pad_sequences(sequences, padding='pre')
print('Shape of Input Sequence:',padded_sequences.shape)
padded_sequences[:5]

```

```

Shape of Input Sequence: (143, 9)
array([[ 0,  0,  0,  0,  0,  0,  0,  0, 52],
       [ 0,  0,  0,  0,  0,  0,  0,  0, 52, 53],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0, 68],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0, 39],
       [ 0,  0,  0,  0,  0,  0,  0,  0, 39, 53]], dtype=int32)

```

Feature Extraction:

```

intent_to_index = {}
categorical_target = []
index = 0

for intent in intents:
    if intent not in intent_to_index:
        intent_to_index[intent] = index
        index += 1
    categorical_target.append(intent_to_index[intent])

num_classes = len(intent_to_index)

```

```
print('Number of Intents :', num_classes)

# Convert intent_to_index to index_to_intent
index_to_intent = {index: intent for intent, index in intent_to_index.items()}
index_to_intent
```

```
Number of Intents : 22
{0: 'Greeting',
 1: 'GreetingResponse',
 2: 'CourtesyGreeting',
 3: 'CourtesyGreetingResponse',
 4: 'CurrentHumanQuery',
 5: 'NameQuery',
 6: 'RealNameQuery',
 7: 'TimeQuery',
 8: 'Thanks',
 9: 'NotTalking2U',
10: 'UnderstandQuery',
11: 'Shutup',
12: 'Swearing',
13: 'GoodBye',
14: 'CourtesyGoodBye',
15: 'WhoAmI',
16: 'Clever',
17: 'Gossip',
18: 'Jokes',
19: 'PodBayDoor',
20: 'PodBayDoorResponse',
21: 'SelfAware'}
```

One-Hot Encoding

```
categorical_vec = tf.keras.utils.to_categorical(categorical_target,
                                                num_classes=num_classes)
categorical_vec = categorical_vec.astype('int32')

print('Shape of Ca', categorical_vec.shape)
categorical_vec[:5]
```

```
Shape of Ca (143, 22)
array([[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]],
      dtype=int32)
```

Model Building:

```
epochs=100
embed_dim=300
lstm_num=50
output_dim=categorical_vec.shape[1]
input_dim=len(unique_intents)
print("Input Dimension : {},\nOutput Dimension : {}".format(input_dim, output_dim))
```

```
Input Dimension :22,
Output Dimension :22
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Embedding(len(tokenizer.word_index) + 1, embed_dim),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(lstm_num, dropout=0.1)),
    tf.keras.layers.Dense(lstm_num, activation='relu'),
    tf.keras.layers.Dropout(0.4),
```

```
tf.keras.layers.Dense(output_dim, activation='softmax')
])

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Training

```
model.fit(padded_sequences, categorical_vec, epochs=epochs, verbose=0)
```

<keras.src.callbacks.history.History at 0x7bebcac7eba0>

Evaluate

```
test_text_inputs = ["Hello",
                    "my name is adam",
                    "how are you?",
                    "can you guess my name?",
                    "Do you get me", "Adios"]

test_intents = ["Greeting",
                "GreetingResponse",
                "CourtesyGreeting",
                "CurrentHumanQuery",
                "UnderstandQuery",
                "GoodBye"]

test_sequences = tokenizer.texts_to_sequences(test_text_inputs)
test_padded_sequences = pad_sequences(test_sequences, padding='pre')
test_labels = np.array([unique_intents.index(intent) for intent in test_intents])
test_labels = tf.keras.utils.to_categorical(test_labels, num_classes=num_classes)
loss, accuracy = model.evaluate(test_padded_sequences, test_labels)
```

1/1 ————— 1s 894ms/step - accuracy: 0.8333 - loss: 0.3106

Predict

```
def response(sentence):
    sent_tokens = []
    # Split the input sentence into words
    words = sentence.split()
    # Convert words to their corresponding word indices
    for word in words:
        if word in tokenizer.word_index:
            sent_tokens.append(tokenizer.word_index[word])
```

```
else:
    # Handle unknown words
    sent_tokens.append(tokenizer.word_index['<unk>'])
sent_tokens = tf.expand_dims(sent_tokens, 0)
#predict numerical category
pred = model(sent_tokens)
#category to intent
pred_class = np.argmax(pred.numpy(), axis=1)
# random response to that intent
return random.choice(
    response_for_intent[index_to_intent[pred_class[0]]], index_to_intent[pred_class[0]]
```

Chatbots: Intent Recognition

```
print("Note: Enter 'quit' to break the loop.")
while True:
    query = input('You: ')
    if query.lower() == 'quit':
        break
    bot_response, typ = response(query)
    print('Geek: {} -- TYPE: {}'.format(bot_response, typ))
    print()
```

```
Note: Enter 'quit' to break the loop.
You: Hi
Geek: That depends, can you prove that you are? -- TYPE: SelfAware

You: and
Geek: That is an difficult question, can you prove that you are? -- TYPE: SelfAware

You: SelfAware
Geek: That is an interesting question, can you prove that you are? -- TYPE: SelfAware

You: quit
```